

Introduction to JavaScript

JavaScript (JS) is a high-level, interpreted programming language used to create **interactive and dynamic web pages**.

- Runs directly in the browser
- Also runs on servers (Node.js)
- Works with HTML and CSS
- Supports object-oriented, functional, and event-driven programming

Initially created to make web pages interactive (buttons, forms, animations, etc.)

History of JavaScript

- Created in **1995** by **Brendan Eich** at **Netscape**
- Originally called **Mocha** → **LiveScript** → **JavaScript**
- Standardized as **ECMAScript (ES)**

Modern JavaScript versions: ES6 (2015) and later

Why JavaScript is Important?

JavaScript is one of the **core technologies of the web**:

1. **HTML** → Structure
2. **CSS** → Design
3. **JavaScript** → Behavior & Interactivity

Features of JavaScript

- Lightweight and fast
- Interpreted (no compilation required)
- Cross-platform
- Event-driven
- Asynchronous programming support
- Prototype-based object orientation
- Case-sensitive language

Where JavaScript Runs

Client-Side (Browser)

Runs inside browsers like:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Used for UI interaction and DOM manipulation.

Server-Side

Using **Node.js**

- Build API
- Handle databases
- Real-time applications
- Backend logic

Ways to Add JavaScript to HTML

JavaScript can be added to an HTML page in **three main ways**:

1. **Inline JavaScript**
2. **Internal JavaScript**
3. **External JavaScript.**

Inline JavaScript

Inline JavaScript is written **directly inside HTML elements** using event attributes. Used for small actions (button click, alerts, etc.)

Syntax

```
<tag event="JavaScript code">
```

Example

```
<button onclick="alert('Hello User')">Click Me</button>
```

✓ Common Event Attributes

Event	Description
onclick	Mouse click
ondblclick	Double click
onmouseover	Mouse hover
onmouseout	Mouse leave
onkeydown	Key pressed
onsubmit	Form submit

Internal JavaScript (Embedded JavaScript)

JavaScript code is written **inside the HTML file** using the `<script>` tag. Suitable for small to medium scripts used only on one page.

Syntax

```
<script>
```

```
JavaScript code
```

```
</script>
```

Example

```
<script>
```

```
    let num1 = 10
```

```
    let num2 = 20
```

```
    console.log(num1 + num2)
```

```
</script>
```

Where to Place Internal Script?

- **Inside `<head>`**
- **Inside `<body>` (Recommended)**

External JavaScript (Best Practice)

JavaScript is written in a separate file with **.js** extension and linked to HTML. Most commonly used method in real-world projects.

Steps to Use External JS

Step 1: Create JS File

 script.js

Step 2: Link JS File to HTML

```
<script src="script.js"></script>
```

Where to Place External Script?

- In `<head>`
- Before Closing `</body>` (Recommended)

Comparison of Three Methods

Feature	Inline	Internal	External
Location	Inside HTML tag	Inside HTML file	Separate JS file
Reusability	No	No	Yes
Maintainability	No	No	Yes
Best for	Tiny scripts	Small projects	Large projects
Performance	Poor	Medium	Best

Variables in JavaScript

A **variable** is a container used to store data values in memory.

Why Variables Are Used?

- Store data
- Reuse values
- Perform calculations
- Handle user input
- Control program flow

Declare Variables in JavaScript

JavaScript provides **three keywords**:

1. **var** — Old method

2. **let** — Modern method
3. **const** — Constant values

var (Old Way)

Syntax

```
var variableName = value;
```

Example

```
var age = 20;
```

Characteristics of var

- Function-scoped
- Can be redeclared
- Can be updated
- Not recommended in modern JS

let (Modern — Recommended)

Introduced in ES6 (2015)

Syntax

```
let variableName = value;
```

Example

```
let marks = 85;
```

Characteristics of let

- Block-scoped
- Can be updated
- Cannot be redeclared in the same scope

- Safer than var
-

const (Constant Variable)

Used for values that should NOT change.

Syntax

```
const variableName = value;
```

Example

```
const PI = 3.14;
```

Characteristics of const

- Block-scoped
- Cannot be updated
- Cannot be redeclared
- Must be initialized during declaration

Key Differences: var vs let vs const

Feature	var	let	const
Scope	Function	Block	Block
Redeclaration	Allowed	Not allowed	Not allowed
Update value	Yes	Yes	No
Modern usage	Avoid	Yes	Yes

Data Types in JavaScript

A **data type** defines the kind of value a variable can hold.

JavaScript is a **dynamically typed language**, so you do NOT need to declare the type explicitly.

Types of Data in JavaScript

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types

Primitive Data Types

Primitive types store **single, simple values** and are immutable (cannot be changed directly).

Primitive Data Type	Example	Result of typeof
Number	let a = 10;	"number"
String	let name = "Ram";	"string"
Boolean	let flag = true;	"boolean"
Undefined	let x;	"undefined"
Null	let data = null;	"object"
Symbol	let id = Symbol("id");	"symbol"
BigInt	let big = 1456789017890n;	"bigint"

typeof null returns "object" due to a historical bug in JavaScript, even though null is a primitive type.

Non-Primitive (Reference) Data Types

Non-Primitive data types store **multiple values or complex data**.

Instead of storing the actual value, they store a **reference (address)** to

the memory location. Mutable (can be changed) Stored by reference

Can hold collections of data More complex than primitive type

Main Non-Primitive Data Types

1. Object
2. Array

Primitive vs Non-Primitive (Comparison)

Feature	Primitive	Non-Primitive
Value	Single	Multiple
Mutability	Immutable	Mutable
Storage	By value	By reference
Examples	Number, String	Object, Array

Array Data Type in JavaScript

An **Array** is a non-primitive (reference) data type used to store **multiple values in a single variable**. Values are stored in an **ordered list** and accessed using an **index**.

- Index starts from **0**
- Can store any data type
- Dynamic (size can change)
- Stored by reference

How to Create an Array?

```
let fruits = ["Apple", "Banana", "Mango"];
```

Accessing Array Elements

Using index number.

```
let fruits = ["Apple", "Banana", "Mango"];  
console.log(fruits[0]); // Apple
```

Modifying Array Elements

```
fruits[1] = "Orange";
```

JavaScript Array Methods

Array Method	Purpose	Example	Output
push()	Add element at end	let a=[1,2]; a.push(3);	[1, 2, 3]
pop()	Remove last element	let a=[1,2,3]; a.pop();	[1, 2]
unshift()	Add element at beginning	let a=[2,3]; a.unshift(1);	[1, 2, 3]
shift()	Remove first element	let a=[1,2,3]; a.shift();	[2, 3]
indexOf()	Find index of element	let a=[10,20,30]; a.indexOf(20);	1
includes()	Check if element exists	let a=[10,20,30]; a.includes(25);	false
slice()	Extract portion (no change to original)	let a=[1,2,3,4]; a.slice(1,3);	[2, 3]
join()	Convert array to string	let a=[1,2,3]; a.join("-");	"1-2-3"
reverse()	Reverse order of elements	let a=[1,2,3]; a.reverse();	[3, 2, 1]
sort()	Sort elements	let a=[3,1,2]; a.sort();	[1, 2, 3]
concat()	Merge arrays	let a=[1,2]; a.concat([3,4]);	[1, 2, 3, 4]
toString()	Convert array to comma string	let a=[1,2,3]; a.toString();	"1,2,3"
toSorted()	Creates a sorted copy of the array without changing the original.	let a = ['A','a','Z']; a.toSorted();	['A','Z','a']
toReversed()	Creates a reversed copy of the array without changing the original	let a = ['A','a','Z']; a.toReversed ();	['Z','a','A']

Object Data Type in JavaScript

An **Object** is a non-primitive (reference) data type used to store **multiple values as key–value pairs**.

- Collection of properties
- Each property has a **key (name)** and **value**
- Values can be any data type
- Stored by reference
- Mutable (can be changed)

Object Syntax

```
let objectName = {  
    key1: value1,  
    key2: value2 };
```

Creating Objects

```
let person = {  
    name: "Shyam",  
    age: 25,  
    isStudent: true};
```

Accessing Object Properties

- **person.name**
- **person["age"]**

Object Built-in Methods

Object Method	Purpose	Example
Object.keys()	Returns an array of all property names (keys)	Object.keys({name:"Ram", age:20})
Object.values()	Returns an array of all property values	Object.values({name:"Ram", age:20})

Object.entries()	Returns an array of [key, value] pairs	Object.entries({name:"Ram", age:20})
Object.hasOwn()	Checks if object has a specific property	Object.hasOwn({a:1}, "a")

String Data Type in JavaScript

A **String** is a sequence of characters used to represent text.

Backticks (Template Literals)

Used for multi-line strings and variable interpolation.

```
let name = "Ritik";
```

```
let s3 = `Hello ${name}`;
```

Characteristics of Strings

- Immutable (cannot be changed directly)
- Indexed (each character has a position)
- Index starts from **0**
- Length property available

Common String Methods

Method	Purpose	Example	Output
toUpperCase()	Convert to uppercase	"ram".toUpperCase()	"RAM"
toLowerCase()	Convert to lowercase	"RAM".toLowerCase()	"ram"
trim()	Remove spaces	" hi ".trim()	"hi"

	from both ends		
slice(start,end)	Extract part of string	"Hello".slice(1,4)	"ell"
substring()	Similar to slice	"Hello".substring(1,4)	"ell"
replace()	Replace text	"Hi Ram".replace("Ram","Shyam")	"Hi Shyam"
includes()	Check if text exists	"Hello".includes("lo")	true
indexOf()	Position of text	"Hello".indexOf("e")	1
split()	Convert string to array	"a,b,c".split(",")	["a","b","c"]
charAt()	Character at index	"Hello".charAt(1)	"e"
concat()	Join strings	"Hi".concat(" Ram")	"Hi Ram"

Operators in JavaScript

Operators are symbols used to perform operations on variables and values.

Arithmetic Operators

Operator	Meaning	Example	Output
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10

/	Division	5 / 2	2.5
%	Modulus (Remainder)	5 % 2	1
**	Exponent (Power)	2 ** 3	8
++	Increment	let a=5; a++;	6
--	Decrement	let a=5; a--;	4

Assignment Operators

Operator	Meaning	Example	Result
=	Assign	x = 10	10
+=	Add & assign	x += 5	x = x + 5
-=	Subtract & assign	x -= 3	x = x - 3
*=	Multiply & assign	x *= 2	x = x * 2
/=	Divide & assign	x /= 2	x = x / 2
%=	Modulus & assign	x %= 3	Remainder

Comparison Operators

Operator	Meaning	Example	Output
==	Equal (value only)	5 == "5"	true
===	Strict equal (value + type)	5 === "5"	false
!=	Not equal	5 != 3	true
!==	Strict not equal	5 !== "5"	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false

>=	Greater or equal	5 >= 5	true
<=	Less or equal	3 <= 5	true

Logical Operators

Operator	Meaning	Example	Output
&&	AND (both true)	true && true	true
&&	AND (both true)	true && false	false
&&	AND (both true)	false && false	false
`	OR (one true)	`true` true	true
`	OR (one true)	true ` false	true
`	OR (one true)	false ` false	false
!	NOT (reverse)	!true	false

Ternary Operator (Conditional)

Short form of if–else.

Syntax:

condition ? value_if_true : value_if_false

Example:

```
let age = 18;
```

```
let result = age >= 18 ? "Adult" : "Minor";
```

Output:

"Adult"

Flow Control Statements in JavaScript

Flow control statements determine **the order in which code executes**.

They allow programs to make decisions, repeat tasks, or jump between blocks of code.

Types of Flow Control Statements

- Conditional Statements (Decision Making)
- Looping Statements (Repetition)
- Jump Statements (Transfer Control)

Category	Statements
Conditional	if, if-else, else-if, switch
Looping	for, while, do-while
Jump	break, continue

Conditional Statements (Decision Making)

Conditional statements allow a program to **make decisions** and execute different code blocks based on conditions.

Used when program behavior depends on input, data, or situation.

Types of Conditional Statements

- if
- if...else
- if...else if...else
- switch
- Ternary Operator (short form)

if Statement

Executes code only when the condition is **true**.

Syntax

```
if (condition) {  
    // code to execute  
}
```

if...else Statement

Chooses between two alternatives.

Syntax

```
if (condition) {  
    // if true  
}  
else {  
    // if false  
}
```

if...else if...else

Used when there are multiple conditions.

Syntax

```
if (condition1) {  
    // block 1  
}  
else if (condition2) {  
    // block 2  
}  
else {
```

```
        // default block  
    }
```

switch Statement

Used when one value needs to be compared with multiple possible values. Works well for menus, options, fixed cases.

Syntax

```
switch (expression) {  
    case value1:  
        // code  
        break;  
    default:  
        // if no case matches  
}
```

Ternary Operator (Conditional Operator)

Short form of if...else.

Syntax

```
condition ? value_if_true : value_if_false
```

Example

```
let age = 18;  
let result = age >= 18 ? "Adult" : "Minor";
```

Looping Statements (Repetition)

Looping statements are used to **repeat a block of code multiple times** until a condition becomes false.

Useful when performing repetitive tasks automatically.

Types of Looping Statements

- **for loop**
- **while loop**
- **do...while loop**

for Loop

Used when the number of iterations is known.

Syntax

```
for (initialization; condition; update) {  
    // code block  
}
```

How it works

- Initialization → runs once
- Condition → checked each time
- Update → changes variable

while Loop

Runs while the condition is true.

Used when number of iterations is not known.

Syntax

```
while (condition) {  
    // code  
}
```

do...while Loop

Executes code **at least once**, even if condition is false.

Syntax

```
do {  
    // code  
} while (condition);
```

Loop Type	Used For	Example Use
for	Known number of repetitions	Print numbers 1–100
while	Unknown repetitions	User input loops
do...while	Execute at least once	Menu systems

Functions in JavaScript

A function is a reusable block of code designed to perform a specific task.

Functions help make programs modular, readable, and reusable.

Defined using the function keyword.

- Avoid code repetition
- Improve readability
- Organize logic
- Easy testing and maintenance
- Reusable code blocks

Basic Syntax

```
function functionName(parameters) {  
    // code block  
    return value; // optional
```

```
}
```

Parameters vs Arguments

Term	Meaning
Parameter	Variable in function definition
Argument	Actual value passed to function

Arrow Function in JavaScript

An Arrow Function is a shorter way to write functions in JavaScript, introduced in ES6 (ECMAScript 2015). Also called: Fat Arrow Function

It uses the => (arrow) symbol and provides a concise syntax.

Why Arrow Functions?

They were introduced to:

- Write shorter function syntax
- Improve readability
- Make functional programming easier

Basic Syntax

Traditional Function

```
function functionName(parameters) {  
    return value;  
}
```

Arrow Function Equivalent

Curly braces {} required when using multiple lines.

```
const functionName = (parameters) => {  
    return value;  
};
```

Simplest Form (One Line)

```
const functionName = (parameters) => value;
```

Examples

Example 1: Simple Addition

Normal Function

```
function add(a, b) {  
    return a + b;  
}
```

Arrow Function

```
const add = (a, b) => {  
    return a + b;  
};
```

Short Form

```
const add = (a, b) => a + b;
```

Immediately Invoked Function Expression (IIFE)

Defined as an expression Executed immediately after creation Runs only once Does not pollute the global scope

```
(function () {  
    console.log("Executed immediately");  
})();
```

Part	Meaning
(function(){})	Function expression (wrapped in parentheses)
()	Immediately invokes the function

Array Functions in JavaScript

forEach() Method

Executes a function for each array element.

Does NOT return a new array

Used for side effects (printing, updating UI, etc.)

Syntax

```
array.forEach(function(element, index, array) {  
    // code  
});
```

Example

```
let nums = [1, 2, 3, 4];
```

```
nums.forEach(function(n) {  
    console.log(n);  
});
```

```
});
```

Using Arrow Function

```
nums.forEach(n => console.log(n));
```

map () Method

Creates a **new array** by applying a function to each element.

Returns a new transformed array

Does NOT change original array

Syntax

```
let newArray = array.map(function(element, index, array) {  
    return newValue;  
});
```

Example

```
let nums = [1, 2, 3, 4];
```

```
let squares = nums.map(n => n * n);
```

```
console.log(squares);
```

Key Difference — map() vs forEach()

Feature	forEach()	map()
Returns value	No	Yes (new array)
Modifies original	No	No
Use case	Side effects	Data transformation
Chainable	No	Yes

filter() Method

Returns elements that satisfy a condition.

```
let nums = [10, 15, 20, 25];  
let result = nums.filter(n => n > 15);  
console.log(result);
```

reduce() Method

Reduces array to a single value.

```
let nums = [1, 2, 3, 4];  
let sum = nums.reduce((total, n) => total + n, 0);  
console.log(sum);
```

find() Method

Returns first element that matches condition.

```
let nums = [5, 10, 15, 20];  
let result = nums.find(n => n > 10);  
console.log(result);
```

some () Method

Checks if at least one element satisfies condition.

```
[1, 3, 5, 8].some(n => n % 2 === 0);
```

every () Method

Checks if all elements satisfy condition.

```
[2, 4, 6].every(n => n % 2 === 0);
```

Method	Purpose	Returns
forEach()	Execute for each element	Nothing
map()	Transform elements	New array
filter()	Select elements	New array
reduce()	Combine values	Single value
find()	First match	Element
some()	Any match	Boolean
every()	All match	Boolean

Synchronous and Asynchronous in JavaScript

JavaScript executes code in two ways:

- **Synchronous (Blocking)**
- **Asynchronous (Non-Blocking)**

Synchronous Programming

In synchronous execution, code runs **line by line**, one after another. Each task must finish before the next task starts.

Characteristics

- Blocking
- Sequential execution
- Simple to understand
- Can cause delays if task is slow

Example

```
console.log("Start");  
console.log("Task 1");  
console.log("Task 2");  
console.log("End");
```

Asynchronous Programming

In asynchronous execution, long tasks run in the background while other code continues. Non-blocking behavior.

Characteristics

- Non-blocking
- Improves performance
- Keeps UI responsive
- Used for time-consuming tasks

Example with setTimeout

```
console.log("Start");  
setTimeout(() => {  
  console.log("Task 1");  
}, 2000);
```

```
console.log("End");
```

Output

Start

End

Task 1

Where Asynchronous Programming is Used

- API calls
- File operations
- Database queries
- Timers
- User input

Asynchronous Techniques in JavaScript

- **Callbacks**
- **Promises**
- **Async/Await**

Synchronous vs Asynchronous

Feature	Synchronous	Asynchronous
Execution	Sequential	Parallel / Background
Blocking	Yes	No
Performance	Lower for heavy tasks	Higher
Complexity	Simple	More complex
Use case	Small tasks	Long operations

Callback Hell in JavaScript

Callback Hell occurs when multiple asynchronous operations are nested inside callbacks, making code difficult to read, maintain, and debug. Also called “**Pyramid of Doom**” because of its deep nested structure.

What is a Callback?

A callback is a function passed as an argument to another function and executed later.

```
function greet(name, callback) {  
    console.log("Hello " + name);
```

```
    callback();  
  }
```

```
  greet("Ram", function () {  
    console.log("Welcome!");  
  });
```

What is Callback Hell?

When callbacks are nested inside callbacks repeatedly.

Example of Callback Hell

```
  setTimeout(() => {  
    console.log("Step 1");  
  
    setTimeout(() => {  
      console.log("Step 2");  
  
      setTimeout(() => {  
        console.log("Step 3");  
  
        setTimeout(() => {  
          console.log("Step 4");  
        }, 1000);  
      }, 1000);  
    }, 1000);  
  }, 1000);
```

```
}, 1000);
```

```
}, 1000);
```

Structure

Step 1

Step 2

Step 3

Step 4

Deep nesting → hard to understand

Why Callback Hell is Bad?

- Poor readability
- Difficult debugging
- Hard maintenance
- Error handling becomes complex
- Code reuse becomes difficult

How to Avoid Callback Hell

Use Named Functions

Instead of anonymous nested functions.

```
function step4() {  
    console.log("Step 4");  
}  
function step3() {  
    console.log("Step 3");  
    setTimeout(step4, 1000);  
}  
function step2() {  
    console.log("Step 2");  
    setTimeout(step3, 1000);  
}  
setTimeout(step2, 1000);
```

Promises in JavaScript

A Promise is an object that represents the result of an asynchronous operation that may complete now, later, or fail. Used to handle asynchronous tasks more cleanly than callbacks.

Why Promises?

- Avoid Callback Hell
- Better readability
- Easier error handling
- Supports chaining
- Foundation for Async/Await

Promise States

A Promise has three states:

State	Meaning
Pending	Initial state (operation not completed)
Fulfilled	Operation completed successfully
Rejected	Operation failed

Creating a Promise

Syntax

```
let promise = new Promise(function(resolve, reject) {  
    // async operation  
});
```

- `resolve()` → success
- `reject()` → failure

Example

```
let p = new Promise((resolve, reject) => {  
    let success = true;  
    if (success) {  
        resolve("Operation Successful");  
    } else {  
        reject("Operation Failed");  
    }  
});
```

Consuming a Promise

Using `.then()` and `.catch()`.

```
p.then(result => {  
    console.log(result);  
}).catch(error => {  
    console.log(error);  
});
```

```
});
```

Promise Chaining

Multiple asynchronous steps in sequence.

```
p.then(result => {  
  console.log(result);  
  return "Step 2";  
})  
.then(step2 => {  
  console.log(step2);  
});
```

Each .then() receives previous result.

Finally Block

Runs regardless of success or failure.

```
p.finally(() => {  
  console.log("Operation completed");  
});
```

Use Async/Await (Best Modern Solution)

Makes asynchronous code look like synchronous code.

```
function HELlo(getid) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log(getid)  
      resolve("success")  
    }, 7000);  
  })  
}
```

```
async function FeatchID() {  
  await HELlo(1)  
  await HELlo(2)  
  await HELlo(3)  
}
```

Real-Life Uses (Where It Appears)

- API request chains
- Event-driven systems
- Authentication flows
- Payment processing

DOM (Document Object Model) in JavaScript

DOM (Document Object Model) is a programming interface for HTML and XML documents.

. It represents the web page as a **tree structure of objects** that JavaScript can read and modify.

Using DOM, JavaScript can:

- Change HTML content
- Change CSS styles
- Add or remove elements
- Handle events (click, input, etc.)
- Validate forms
- Create dynamic web pages

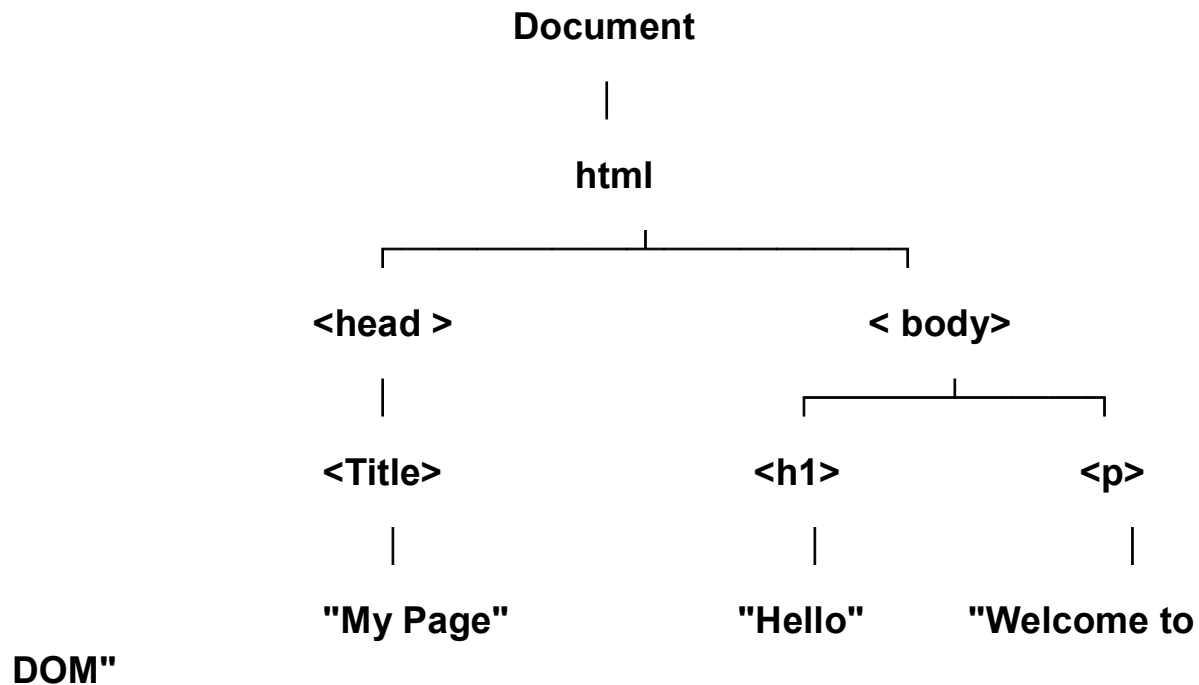
Real-Life Meaning

Think of DOM as a **live map of your webpage**.

HTML → Structure

DOM → Interactive version of that structure for JavaScript

DOM Tree Structure



Types of Nodes

Node Type	Example
Document Node	entire page
Element Node	<h1>, <p>
Text Node	"Hello"
Attribute Node	class, id

Important Relationships

DOM works like a family tree

- **Parent Node** → body is parent of h1 and p
- **Child Node** → h1 is child of body
- **Sibling Node** → h1 and p are siblings

Why DOM Tree is Important

JavaScript uses this tree to:

- Find elements
- Change content
- Add/remove nodes
- Handle events
- Build dynamic pages

DOM Methods in JavaScript

DOM Methods are functions provided by JavaScript to interact with the web page. They allow you to **select, create, modify, and delete HTML elements**

Methods to Select Elements

`getElementById()`

Selects element by ID (returns single element)

```
<h1 id="title">Hello</h1>
```

```
let el = document.getElementById("title");
```

```
el.style.color = "red";
```

`getElementsByClassName()`

Selects elements by class name

```
<p class="msg">Hi</p>
```

```
<p class="msg">Hello</p>
```

```
let items =
```

```
document.getElementsByClassName("msg");
```

```
items[0].style.color = "blue";
```

Returns HTMLCollection (array-like)

`getElementsByTagName()`

Selects elements by tag name

```
<p>First</p>
```

```
<p>Second</p>
```

```
let ritik = document.getElementsByTagName("p");  
ritik[1].style.color = "blue";
```

`querySelector()` (Modern & Powerful)

Selects first matching element (CSS selector)

```
<p class="demo">Text 1</p>
```

```
<p class="demo">Text 2</p>
```

```
let el = document.querySelector(".demo");  
el.style.fontSize = "30px";
```

`querySelectorAll()`

Selects all matching elements

```
let all = document.querySelectorAll(".demo");  
all[1].style.color = "green";
```

Methods to Change Content

`innerHTML`

```
<p id="demo"></p>
```

```
document.getElementById("demo").innerHTML =  
"<b>Hello</b>";
```

Adds HTML inside element

`innerText`

```
<p id="text"></p>
```

```
document.getElementById("text").innerText = "Hello";
```

Only visible text

```
textContent
```

```
document.getElementById("text").textContent = "Hello";
```

Includes hidden text too

Methods to Change Attributes

```
getAttribute()
```

```
<a id="link" href="url">Visit</a>
```

```
let link = document.getElementById("link");
```

```
console.log(link.getAttribute("href"));
```

```
setAttribute()
```

```
link.setAttribute("href", "https://google.com");
```

```
removeAttribute()
```

```
<a id="link" href="#" target="_blank">Visit</a>
```

```
link.removeAttribute("target");
```

```
classList.add()
```

```
<p id="box">Text</p>
```

```
document.getElementById("box").classList.add("active");
```

```
classList.remove()
```

```
document.getElementById("box").classList.remove("active");
```

Methods to Create Elements

```
createElement()  
  
    let p = document.createElement("p");  
    p.innerText = "New Paragraph";  
  
appendChild()  
  
    document.body.appendChild(p);
```

Methods to Remove Elements

```
remove()  
  
    <p id="removeMe">Delete me</p>  
  
    document.getElementById("removeMe").remove();  
  
removeChild()  
  
    <ul id="myList">  
        <li id="item1">Item 1</li>  
    </ul>  
  
    let parent = document.getElementById("myList");  
    let child = document.getElementById("item1");  
    parent.removeChild(child);
```

Summary Table

Category	Important Methods
----------	-------------------

Selecting	getElementById, querySelector
Content	innerHTML, innerText
Attributes	getAttribute, setAttribute
Styles	style, classList
Create	createElement, appendChild
Remove	remove, removeChild

DOM is Important

Without DOM:

- Web pages would be static
- No interactivity
- No dynamic updates

With DOM:

- Interactive websites
- Real-time updates
- Modern web apps

DOM vs HTML

HTML	DOM
Static structure	Dynamic object model
Written by developer	Controlled by JavaScript
Does not change automatically	Can change anytime

addEventListener in JavaScript

Event

An **event** is any action that happens in the browser.

- User actions
- Browser actions

Examples of Events

- Mouse click
- Key press
- Page load
- Form submit
- Mouse hover
- Input typing

addEventListener

addEventListener is a DOM method used to **attach an event handler to an element**. When this event happens, run this function.

Why Use addEventListener() ?

- Cleaner than inline events (onclick="")
- Allows multiple handlers
- Better control
- Separates HTML and JS
- Industry best practice

Syntax

```
element.addEventListener(event, function);
```

Parameter	Description
event	Event name (without "on")

function	Function to execute
----------	---------------------

Basic Example

HTML

```
<button id="btn">Click Me</button>
```

JavaScript

```
let btn = document.getElementById("btn")
```

```
btn.addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

How It Works

- User clicks button
- Browser detects event
- Event listener runs function
- Action performed

Types of Event Handlers

Using Anonymous Function

```
btn.addEventListener("click", function() {  
    console.log("Clicked");  
});
```

Using Named Function

```
function sayHello() {  
    alert("Hello!");  
}
```

```
}
```

```
btn.addEventListener("click", sayHello);
```

Arrow Function

```
btn.addEventListener("click", () => {  
    alert("Arrow function");  
});
```

Multiple Events on Same Element

```
btn.addEventListener("mouseover", () => {  
    console.log("Mouse over");  
});
```

```
btn.addEventListener("mouseout", () => {  
    console.log("Mouse out");  
});
```

Common DOM Events

Event	Description
click	Mouse click
dblclick	Double click

mouseover	Mouse enters element
mouseout	Mouse leaves element

addEventListener vs onclick

Feature	addEventListener	onclick
Multiple handlers	Yes	No
Separation of code	Yes	No
Best practice	Yes	No

BOM (Browser Object Model) in JavaScript

BOM (Browser Object Model) allows JavaScript to interact with the **browser window and browser features**, not just the webpage content.

BOM = Interface between JavaScript and the Browser

It controls things outside the HTML document:

- Browser window
- URL
- Tabs & navigation
- History
- Screen info
- Dialog boxes
- Browser details

BOM Hierarchy

window (Top Object)

- |—— document → DOM (Webpage)
- |—— location → URL info
- |—— history → Navigation history
- |—— navigator → Browser info
- |—— screen → Screen details

Everything in BOM starts from the **window object**

window Object (Global Object)

Represents the browser window/tab.

All global JavaScript objects, functions, and variables automatically become members of window.

Window Object :

The window object represents a window in browser. An object of window is created automatically by the browser. Window is the object of browser, it is not the object of javascript. The javascript objects are string, array, date etc.

Methods of window object

alert() :

displays the alert box containing message with ok button.

confirm() :

displays the confirm dialog box containing message with ok

and cancel button.

prompt() :

displays a dialog box to get input from the user.

open() :

opens the new window.

**<input type="button" value="Google"
onclick="Open()"/>**

<script>

function Open(){

open('http://www.google.com');

}

</script>

setTimeout() :

JavaScript that allows you to execute a function or a block of code after a specified delay (in milliseconds).

function greet(name) {

```
        return name;
    }

    let result = greet('Ritik');

    setTimeout ( () => {
        document.write(result);
    }, 3000);
```

JavaScript Navigator Object

The JavaScript navigator object is used for browser detection. It can be used to get browser information .

appCodeName :

Returns the code name of the browser (e.g., "Mozilla")

```
console.log(navigator.appCodeName);
```

appName :

Returns the name of the browser (e.g., "Netscape").

```
Console.log(navigator.appName);
```

appVersion :

Returns the version information of the browser.

```
Console.log (navigator.appVersion);
```

cookieEnabled :

Indicates whether cookies are enabled in the browser (true or false).

```
Console.log (navigator.cookieEnabled);
```

language :

Returns the browser's language (e.g., "en-US").

Console.log (navigator.language);

userAgent :

Returns the browser's user agent string, which includes information about the browser, operating system, and device

Console.log (navigator.userAgent);

platform :

Returns the platform on which the browser is running (e.g., "Win32", "MacIntel").

Console.log (navigator.platform);

JavaScript Screen Object :

The JavaScript screen object holds information of browser screen. It can be used to display screen width, height, colorDepth, pixelDepth etc.

width :

Console.log (screen.width);

height :

Console.log (screen.height);

availWidth :

Console.log (screen.availWidth);

availHeight :

Console.log (screen.availHeight);

colorDepth :

Returns the number of bits used to display one color.

Console.log (screen.colorDepth);

pixelDepth :

Returns the number of bits used for a single pixel.

Console.log (screen.pixelDepth);

popup Windows Box

Alert:

Purpose: To display a simple message to the user.

Interaction: Shows a message with an "OK" button.

Returns: No return value (doesn't expect user input).

Prompt:

Purpose: To ask for user input.

Interaction: Shows a text box along with "OK" and "Cancel"

Returns: The input text if "OK" is clicked, null if "Cancel" is click.

Confirm:

Purpose: To ask for confirmation from the user (e.g., Yes/No, OK/Cancel).

Interaction: Shows "OK" and "Cancel" buttons.

Returns: true if "OK" is clicked, false if "Cancel" is clicked.

BOM vs DOM

Feature	BOM	DOM
Focus	Browser	Webpage content
Controls	Window, URL	HTML elements
Top Object	window	document